RM - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continous color palettes
```

```
# load needed functions
# source(paste0(RootDir,"/src/function_library/sp_functions.R"))
# do this in the calling R script
```

```
# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste(CurPat," - ",CurTask," - ",RunLabel," - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE
```

```
CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres
```

```
# done in calling script
# print(paste0("Using root dir: ",RootDir))
# RunDir <- paste0(paste0(RootDir,"/output/",RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir,showWarnings = TRUE)
#   print(paste0("created run directory: ",RunDir))
# }
```

```
# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir,showWarnings = TRUE)
#   print(paste0("created participant directory: ",PatientDir))
# }
# TablesDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat,"/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir,showWarnings = TRUE)
#   print(paste0("created tables directory: ",TablesDir))
# }
# FigDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat,"/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir,showWarnings = TRUE)
#   print(paste0("created figures directory: ",FigDir))
# }
# ReportsDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat,"/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir,showWarnings = TRUE)
#   print(paste0("created reports directory: ",ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())
```

```
ModelDatFilename<-paste0(RootDir,"/data/",CurPat,"/",CurPat,"_",CurTask,"_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ",ModelDatFilename))
  print(sprintf("\troot dir: ",RootDir))
}
PosDat<-read.csv(ModelDatFilename)
```

```
# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values",OrigDataLen))
```

```
## [1] "Original data has 4444 values"
```

```
if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values",DataLen))
  print(sprintf("%d values eliminated.",OrigDataLen - DataLen))
}
```

```
## [1] "Data without fractional preserved values has 4398 values"
## [1] "46 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite

# get counts of syllable components in different serial positions
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syl

syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))

kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	555	36	133	NA	NA	724
2	68	NA	442	100	112	722
3	318	NA	171	210	16	715
4	305	NA	241	71	39	656
5	239	NA	210	70	37	556
6	206	1	136	71	23	437
7	179	NA	103	29	19	330
8	91	NA	54	25	4	174
9	75	NA	2	NA	7	84

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7665746	0.0497238	0.1837017	NA	NA	724
2	0.0941828	NA	0.6121884	0.1385042	0.1551247	722
3	0.4447552	NA	0.2391608	0.2937063	0.0223776	715
4	0.4649390	NA	0.3673780	0.1082317	0.0594512	656
5	0.4298561	NA	0.3776978	0.1258993	0.0665468	556
6	0.4713959	0.0022883	0.3112128	0.1624714	0.0526316	437
7	0.5424242	NA	0.3121212	0.0878788	0.0575758	330
8	0.5229885	NA	0.3103448	0.1436782	0.0229885	174
9	0.8928571	NA	0.0238095	NA	0.0833333	84

```
# get percentages only from summary table
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
syll_comp_perc <- syll_comp_perc %>% rename("Coda" ="1","Satellite"="S")

# pivot to long format for plotting
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
            names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda","Sa
```

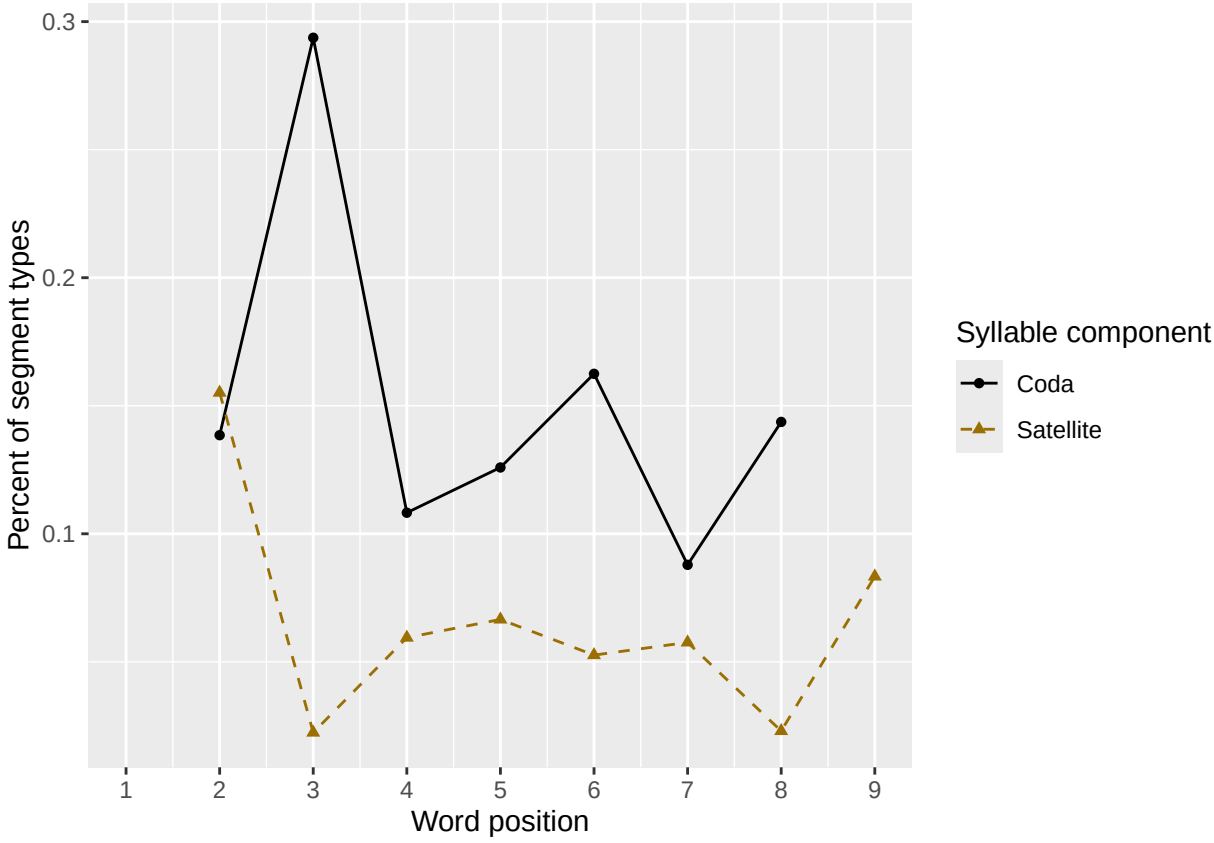
```
# plot satellite and coda occurances across position

syll_comp_plot <- ggplot(syll_comp_plot_data,
                         aes(x=pos,y=percent,group=syll_component,
                             color=syll_component,
                             linetype = syll_component,
                             shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype_
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_pos_of_syll_complexities_in_stimuli.png"),plot=sy
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`)
```

```
syll_comp_plot
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```




```
# len/pos table
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```
min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,"_preserved_percentages_by_len_
pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`    `4`    `5`    `6`    `7`    `8`    `9`
##     <int> <dbl> <dbl> <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1       4 0.952 0.984 0.935 NA     NA     NA     NA     NA     NA
## 2       5 0.979 0.979 0.990  0.948 NA     NA     NA     NA     NA
## 3       6 0.967 0.958 0.925  0.941  0.908 NA     NA     NA     NA
## 4       7 0.982 1     0.946  0.929  0.920  0.851 NA     NA     NA
## 5       8 0.974 0.981 0.980  0.941  0.934  0.940  0.915 NA     NA
## 6       9 0.956 0.967 0.922  0.868  0.820  0.865  0.815  0.791 NA
## 7      10 0.988 0.965 0.988  0.917  0.916  0.843  0.835  0.867  0.821
```

```
# len/pos table
pos_len_N <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved),N=n()) %>% dply
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                 "_N_by_len_position.csv"))
pos_len_N_table
```

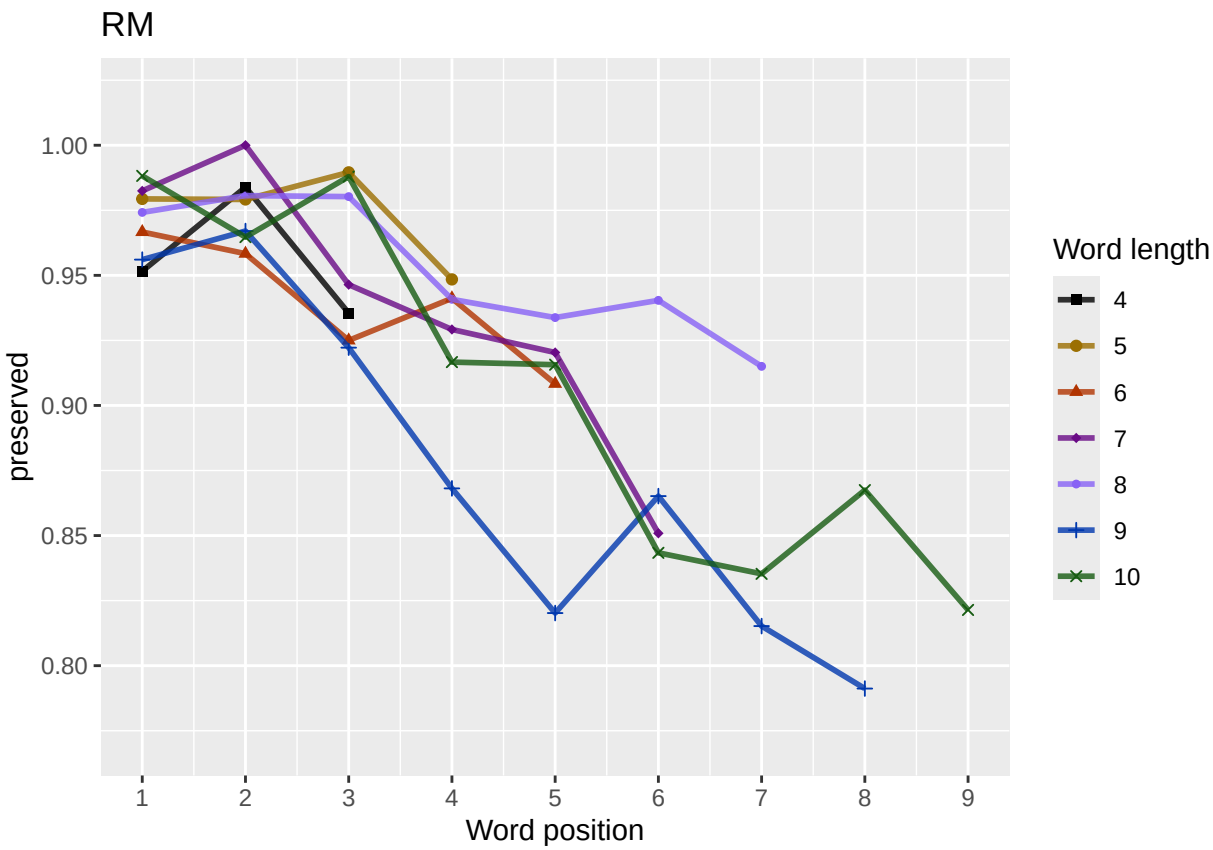
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##     <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1       4    62    62    62    NA    NA    NA    NA    NA    NA
## 2       5    97    96    96    97    NA    NA    NA    NA    NA
## 3       6   120   120   120   119   120    NA    NA    NA    NA
## 4       7   114   113   112   113   113   114    NA    NA    NA
## 5       8   155   155   152   152   151   151   153    NA    NA
## 6       9    91    91    90    91    89    89    92    91    NA
## 7      10    85    85    83    84    83    83    85    83    84
```

```
obs_linetypes <- c("solid","solid","solid","solid",
                   "solid","solid","solid","solid","solid")
pred_linetypes <- "dashed"
pos_len_summary$stimlen<-factor(pos_len_summary$stimlen)
pos_len_summary$pos<-factor(pos_len_summary$pos)
# len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=stimle
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
                                           paste0(CurPat),
                                           NULL,
                                           c(min_preserved,max_preserved),
                                           palette_values,
                                           shape_values,
                                           obs_linetypes,
                                           pred_linetypes)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot,device="png",unit="cm",width=15,height=11)
len_pos_plot
```



Length and position

```
# length and position

LPModelEquations<-c("preserved ~ 1",
            "preserved ~ stimlen",
            "preserved ~ pos",
            "preserved ~ stimlen + pos",
            "preserved ~ stimlen*pos",
            "preserved ~ I(pos^2)+pos",
            "preserved ~ stimlen + I(pos^2) + pos",
            "preserved ~ stimlen * (I(pos^2) + pos)"
)

LPRes<-TestModels(LPModelEquations,PosDat)
```

```
## ****************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
```

```
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     4.33269      0.02188     -0.51415
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       2237
## Residual Deviance: 2110  AIC: 2116
## log likelihood:  -1055.033
## Nagelkerke R2:  0.07130458
## % pres/err predicted correctly:  -556.323
## % of predictable range [ (model-null)/(1-null) ]:  0.03003384
## ***************************
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
##     4.76423     -0.05990      0.02482     -0.52100
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4394 Residual
## Null Deviance:       2237
## Residual Deviance: 2108  AIC: 2116
## log likelihood:  -1054.141
## Nagelkerke R2:  0.07229372
## % pres/err predicted correctly:  -555.986
## % of predictable range [ (model-null)/(1-null) ]:  0.03062034
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      3.8999      -0.2978
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       2237
## Residual Deviance: 2113  AIC: 2117
## log likelihood:  -1056.663
## Nagelkerke R2:  0.06949819
## % pres/err predicted correctly:  -556.1426
## % of predictable range [ (model-null)/(1-null) ]:  0.03034777
## ***************************
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          I(pos^2)               pos  stimlen:I(pos^2)       stiml
```

```
##         3.034795          0.172878         -0.034641          0.213337          0.008023         -0.
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4392 Residual
## Null Deviance:       2237
## Residual Deviance: 2106  AIC: 2118
## log likelihood:  -1053.005
## Nagelkerke R2:  0.07355191
## % pres/err predicted correctly:  -555.5989
## % of predictable range [ (model-null)/(1-null) ]:  0.03129398
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##     4.17076     -0.04346     -0.28168
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       2237
## Residual Deviance: 2112  AIC: 2118
## log likelihood:  -1056.176
## Nagelkerke R2:  0.07003799
## % pres/err predicted correctly:  -555.8985
## % of predictable range [ (model-null)/(1-null) ]:  0.0307726
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##    4.411723    -0.072030    -0.344848     0.007248
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4394 Residual
## Null Deviance:       2237
## Residual Deviance: 2112  AIC: 2120
## log likelihood:  -1056.104
## Nagelkerke R2:  0.07011737
## % pres/err predicted correctly:  -555.9702
## % of predictable range [ (model-null)/(1-null) ]:  0.03064785
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.2860      -0.2154
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
```

```
## Null Deviance:       2237
## Residual Deviance: 2202  AIC: 2206
## log likelihood:  -1101.005
## Nagelkerke R2:  0.01982059
## % pres/err predicted correctly:  -569.0766
## % of predictable range [ (model-null)/(1-null) ]:  0.00783731
## ****************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       2.583
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4397 Residual
## Null Deviance:       2237
## Residual Deviance: 2237  AIC: 2239
## log likelihood:  -1118.451
## Nagelkerke R2:  -5.56958e-16
## % pres/err predicted correctly:  -573.5798
## % of predictable range [ (model-null)/(1-null) ]:  0
## ****************************
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                         AIC=LPRes$AIC,
                         row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                              "_len_pos_model_summary.csv"),row.names = FALSE)
kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ I(pos^2) + pos	2116.067	0.0000000	1.0000000	0.3072307	0.0713046	2.332694	NA	-0.5141548	NA	0.0218799	NA
preserved ~ stimlen + I(pos^2) + pos	2116.281	1.2145600	0.8982798	0.2759782	0.0722947	2.764231	-0.0599038	-0.5209953	NA	0.0248159	NA
preserved ~ pos	2117.325	5.2587260	0.5329302	0.1637383	0.0694982	2.899862	NA	-0.2977816	NA	NA	NA
preserved ~ stimlen * (I(pos^2) + pos)	2118.009	9.9424498	0.3786190	0.1163287	0.0735539	3.0347950	0.1728781	0.2133367	-0.0979310	-0.0346408	0.0080234

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen + pos	2118.352	2.2851924	0.3189898	0.0980038	0.0700384	4.0170764	-	-	NA	NA	NA
							0.0434578	0.2816778			
preserved ~ stimlen * pos	2120.209	4.1420070	0.1260592	0.0387294	0.0701174	4.4411723	-	-	0.0072483	NA	NA
							0.0720308	0.3448481			
preserved ~ stimlen	2206.018	89.9437688	0.0000000	0.0000000	0.0198246	4.6286008	-	NA	NA	NA	NA
							0.2154392				
preserved ~ 1	2238.901	122.8345942	0.0000000	0.0000000	0.0000000	4.2582714	NA	NA	NA	NA	NA

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ I(pos^2) + pos"
```

```
print(BestLPModel)
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     4.33269      0.02188     -0.51415
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       2237
## Residual Deviance: 2110  AIC: 2116
```

```
PosDat$LPFitted<-fitted(BestLPModel)
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],
                        NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
fitted_pos_len_table
```

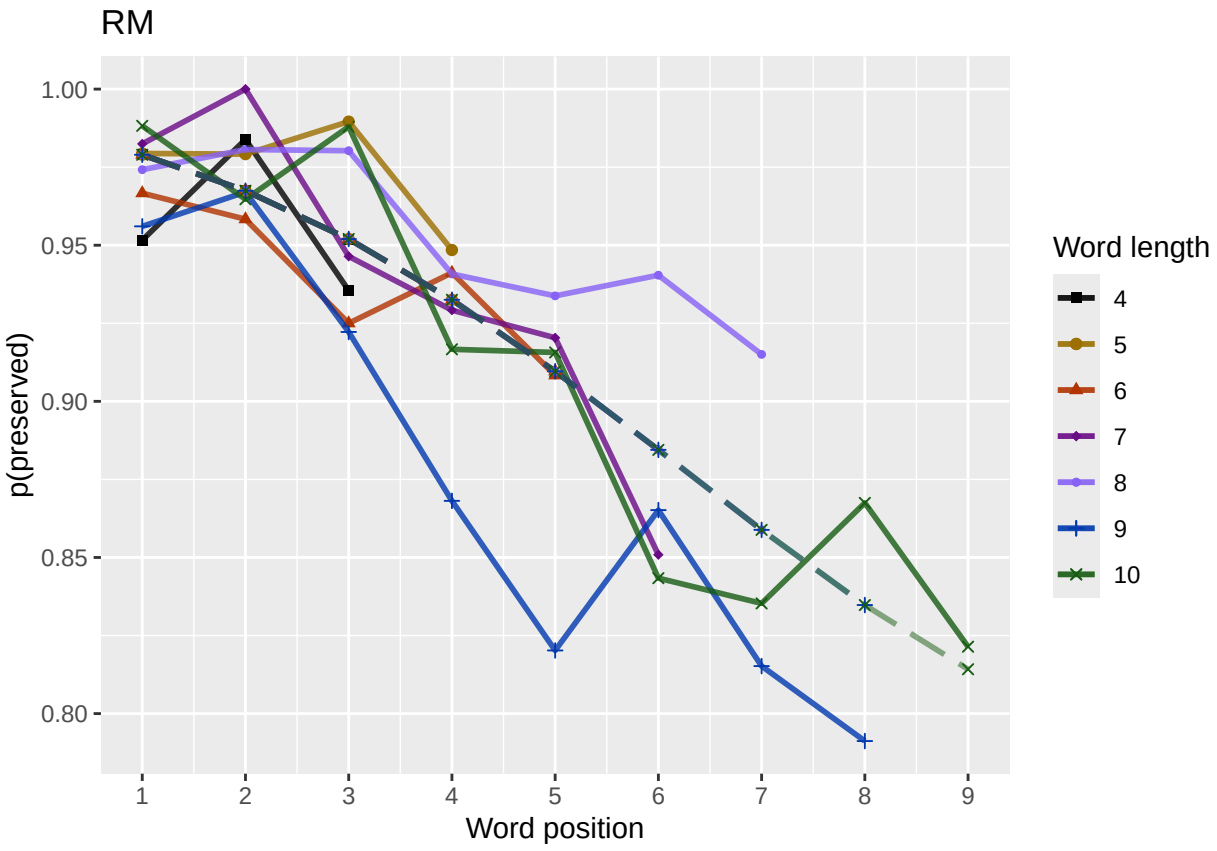
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`    `4`    `5`    `6`    `7`    `8`    `9`
##     <int> <dbl> <dbl> <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1       4 0.979 0.967 0.952 NA     NA     NA     NA     NA     NA
## 2       5 0.979 0.967 0.952  0.933 NA     NA     NA     NA     NA
## 3       6 0.979 0.967 0.952  0.933  0.910 NA     NA     NA     NA
## 4       7 0.979 0.967 0.952  0.933  0.910  0.884 NA     NA     NA
## 5       8 0.979 0.967 0.952  0.933  0.910  0.884  0.859 NA     NA
## 6       9 0.979 0.967 0.952  0.933  0.910  0.884  0.859  0.835 NA
## 7      10 0.979 0.967 0.952  0.933  0.910  0.884  0.859  0.835  0.814
```

```
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
```

```
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitte
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(pas

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
                                          paste0(PosDat$patient[1]),
                                          "LPFitted",
                                          NULL,
                                          palette_values,
                                          shape_values,
                                          obs_linetypes,
                                          pred_linetypes = c("longdash")
                                          )
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_percent_preserved_by_length_pos_wfit.png"),plot=
fitted_len_pos_plot
```



length and position without fragments to see if this changes position^2 influence

```
# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models

# number responses
resp_num<-0
```

```
prev_pos<-9999 # big number to initialize (so first position is smaller)
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array
```

```
# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag
```

```
## # A tibble: 1 x 2
##   frag_sum     N
##      <int> <int>
## 1       64   725
```

```
num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100
```

```
## Warning: Unknown or uninitialised column: `percent_with_frag`.
```

```
print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
              num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))
```

```
## [1] "The number of responses with fragments was 64 / 725 = 8.83 percent"
```

```
write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_percent_fragments.csv"),row.names = FALSE)
```

```
# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)
```

```
## ****************************
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen         I(pos^2)              pos  stimlen:I(pos^2)       stiml
##          1.68388           0.36173          -0.05398           1.02636           0.01698          -0
##
## Degrees of Freedom: 4237 Total (i.e. Null);  4232 Residual
## Null Deviance:       1290
## Residual Deviance: 1267  AIC: 1279
## log likelihood:  -633.4913
## Nagelkerke R2:  0.02096855
## % pres/err predicted correctly:  -284.6735
## % of predictable range [ (model-null)/(1-null) ]:  0.006432137
## ****************************
```

```
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
##     5.18122     -0.12164      0.06061     -0.53687
##
## Degrees of Freedom: 4237 Total (i.e. Null);  4234 Residual
## Null Deviance:       1290
## Residual Deviance: 1276  AIC: 1284
## log likelihood:  -637.835
## Nagelkerke R2:  0.01319433
## % pres/err predicted correctly:  -285.4279
## % of predictable range [ (model-null)/(1-null) ]:  0.003808219
## **************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     4.28051      0.05401     -0.51591
##
## Degrees of Freedom: 4237 Total (i.e. Null);  4235 Residual
## Null Deviance:       1290
## Residual Deviance: 1281  AIC: 1287
## log likelihood:  -640.3069
## Nagelkerke R2:  0.00876295
## % pres/err predicted correctly:  -285.8706
## % of predictable range [ (model-null)/(1-null) ]:  0.00226851
## **************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.1336      -0.1059
##
## Degrees of Freedom: 4237 Total (i.e. Null);  4236 Residual
## Null Deviance:       1290
## Residual Deviance: 1286  AIC: 1290
## log likelihood:  -642.9443
## Nagelkerke R2:  0.004029334
## % pres/err predicted correctly:  -286.2084
## % of predictable range [ (model-null)/(1-null) ]:  0.001093897
## **************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##     4.13629     -0.09467     -0.02338
##
## Degrees of Freedom: 4237 Total (i.e. Null);  4235 Residual
## Null Deviance:       1290
## Residual Deviance: 1286  AIC: 1292
## log likelihood:  -642.7845
## Nagelkerke R2:  0.004316286
## % pres/err predicted correctly:  -286.189
## % of predictable range [ (model-null)/(1-null) ]:  0.001161139
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       3.312
##
## Degrees of Freedom: 4237 Total (i.e. Null);  4237 Residual
## Null Deviance:       1290
## Residual Deviance: 1290  AIC: 1292
## log likelihood:  -645.1866
## Nagelkerke R2:  4.229567e-16
## % pres/err predicted correctly:  -286.5229
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##     3.51139     -0.05197
##
## Degrees of Freedom: 4237 Total (i.e. Null);  4236 Residual
## Null Deviance:       1290
## Residual Deviance: 1289  AIC: 1293
## log likelihood:  -644.3033
## Nagelkerke R2:  0.001587871
## % pres/err predicted correctly:  -286.4148
## % of predictable range [ (model-null)/(1-null) ]:  0.0003760729
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen          pos  stimlen:pos
##     3.84985     -0.06037      0.06934     -0.01077
##
## Degrees of Freedom: 4237 Total (i.e. Null);  4234 Residual
## Null Deviance:       1290
## Residual Deviance: 1285  AIC: 1293
## log likelihood:  -642.7041
## Nagelkerke R2:  0.004460587
## % pres/err predicted correctly:  -286.1788
## % of predictable range [ (model-null)/(1-null) ]:  0.001196762
## ***************************
```

```
NoFragBestLPModel<-NoFrag_LPRes$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPRes$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPRes$Model,
                       AIC=NoFrag_LPRes$AIC,
                       row.names = NoFrag_LPRes$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPRes$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPRes$CoefficientValues, by='row.names',sort=FALSE
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	1278.983	0.000000	1.000000	0.8871465	0.0209681	5.6838750	-0.3617311	1.0263644	-0.2176209	-0.0539763	0.016984
preserved ~ stimlen + I(pos^2) + pos	1283.670	4.687282	0.095977	0.0851461	0.0131945	3.3181222	-0.1216406	0.5368694	NA	0.0606074	NA
preserved ~ I(pos^2) + pos	1286.617	7.631155	0.022025	0.0195394	0.0087624	4.2280510	NA	0.5159149	NA	0.0540144	NA
preserved ~ stimlen	1289.889	10.905859	0.004288	0.0038030	0.0040294	3.3133577	-0.1058758	NA	NA	NA	NA
preserved ~ stimlen + pos	1291.569	12.586293	0.001848	0.0016400	0.0043163	3.1136288	-0.0946678	0.0233840	NA	NA	NA
preserved ~ 1	1292.373	13.390610	0.001236	0.0010971	0.0000000	3.0312109	NA	NA	NA	NA	NA
preserved ~ pos	1292.607	13.623836	0.001100	0.0009764	0.0015837	2.9511394	NA	0.0519689	NA	NA	NA
preserved ~ stimlen * pos	1293.408	14.425582	0.000737	0.0006538	0.0044636	3.0849852	-0.0603652	0.0693412	-0.0107660	NA	NA

```
# plot no fragment data

NoFragData$LPFitted<-fitted(NoFragBestLPModel)
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
                          NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

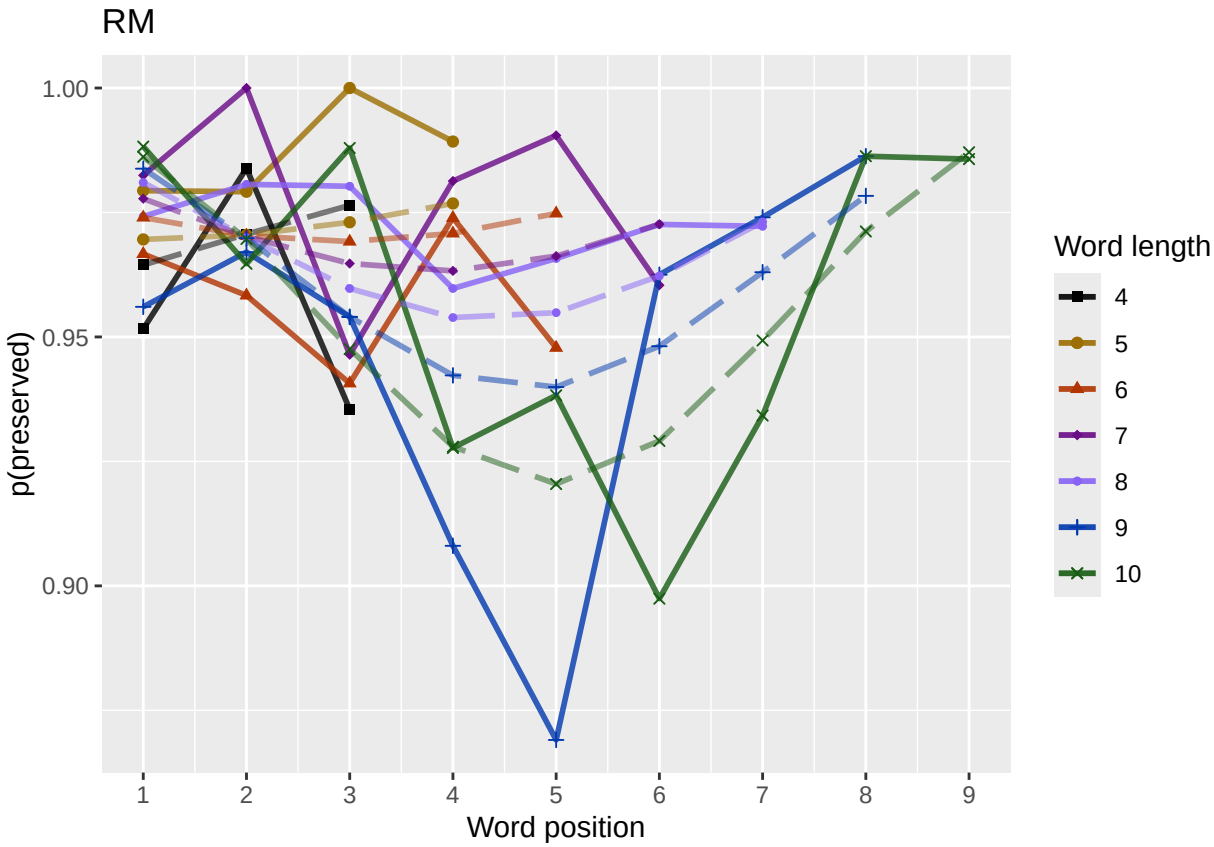
```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`    `4`    `5`    `6`    `7`    `8`    `9`
##     <int> <dbl> <dbl> <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1       4 0.964 0.971 0.976 NA     NA     NA     NA     NA     NA
## 2       5 0.970 0.970 0.973  0.977 NA     NA     NA     NA     NA
## 3       6 0.974 0.970 0.969  0.971  0.975 NA     NA     NA     NA
## 4       7 0.978 0.970 0.965  0.963  0.966  0.973 NA     NA     NA
## 5       8 0.981 0.970 0.960  0.954  0.955  0.962  0.973 NA     NA
## 6       9 0.984 0.970 0.954  0.942  0.940  0.948  0.963  0.978 NA
## 7      10 0.986 0.970 0.948  0.928  0.920  0.929  0.949  0.971  0.987
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitte
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(pas

nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
                                        paste0(NoFragData$patient[1]),
                                        "LPFitted",
                                        NULL,
                                        palette_values,
                                        shape_values,
                                        obs_linetypes,
                                        pred_linetypes = c("longdash")
                                        )
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
              "no_fragments_percent_preserved_by_length_pos_wfit.png"),
       plot=nofrag_fitted_len_pos_plot,
       device="png",unit="cm",
       width=15,height=11)
nofrag_fitted_len_pos_plot
```



back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.77 - 1.02"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account

# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
table_to_use <- fitted_pos_len_table
# take away column with length information
table_to_use <- table_to_use[,2:ncol(table_to_use)]

# take averages for positions
# don't want downward estimates influenced by return upward of U
# therefore, for downward influence, use only the values before the min
# take the difference between each value (differences between position proportion correct) **NOTE** pro
# average the difference in probabilities

# in case min is close to the end or we are not using a min (for non-U-shaped)
# functions, we need to get differences _first_ and then average those (e.g.
# if there is an effect of length and we just average position data,
# later positions) will have a lower average (because of the length effect)
# even if all position functions go upward
```

```
table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)
```

```
## [1] "mean change in probability for each additional length: "
```

```
print(OA_mean_len_diff)
```

```
## [1] 0
```

```
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding  U-shape): "
print(CurrentLabel)
```

```
## [1] "mean change in probability for each additional position (excluding  U-shape): "
```

```
print(OA_mean_pos_diff)
```

```
## [1] -0.02059514
```

```
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)
```

```
if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
                               2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
```

```
    CurrentLabel<-"Average upward change after U minimum"
    print(CurrentLabel)
    print(OA_mean_pos_u_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

    CurrentLabel<-"Proportion of average downward change"
    prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
  }
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}
```

```
## [1] "No U-shape in this participant"
```

```
if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upwa
```

```
  CurrentLabel<-"return upward value"
  print(upward_dist[biggest_return_upward_row])
   results_report_DF <- AddReportLine(results_report_DF,
                                      CurrentLabel,
                                      upward_dist[biggest_return_upward_row])

  print(" ")
  # percentage return upward
  percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upwar
  CurrentLabel <- "Return upward as a proportion of the downward distance:"
  print(CurrentLabel)
  print(percentage_return_upward)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                         percentage_return_upward)
}else{
  print("no U-shape in this participant")
}
```

```
## [1] "no U-shape in this participant"
```

```
FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small
            "preserved ~ stimlen*log_freq",
            "preserved ~ stimlen+log_freq",
            "preserved ~ pos*log_freq",
            "preserved ~ pos+log_freq",
            "preserved ~ stimlen*log_freq + pos*log_freq",
            "preserved ~ stimlen*log_freq + pos",
            "preserved ~ stimlen + pos*log_freq",
            "preserved ~ stimlen + pos + log_freq",
            "preserved ~ (I(pos^2)+pos)*log_freq",
            "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
            "preserved ~ stimlen*log_freq + I(pos^2) + pos",
            "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
            "preserved ~ stimlen + I(pos^2) + pos + log_freq",

            # models without frequency
            "preserved ~ 1",
            "preserved ~ stimlen",
            "preserved ~ pos",
            "preserved ~ stimlen + pos",
            "preserved ~ stimlen*pos",
            "preserved ~ I(pos^2)+pos",
            "preserved ~ stimlen + I(pos^2) + pos",
            "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)
```

```
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
```

```
## Coefficients:
## (Intercept)          pos     log_freq
##      3.8866      -0.2801       0.1935
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       2237
## Residual Deviance: 2077  AIC: 2083
## log likelihood:  -1038.554
## Nagelkerke R2:  0.08949937
## % pres/err predicted correctly:  -550.1509
## % of predictable range [ (model-null)/(1-null) ]:  0.04077576
## ***************************
## model index:  13
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos     log_freq
##    4.351871    -0.000747     0.023469    -0.511134     0.194499
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4393 Residual
## Null Deviance:       2237
## Residual Deviance: 2073  AIC: 2083
## log likelihood:  -1036.705
## Nagelkerke R2:  0.09153272
## % pres/err predicted correctly:  -550.3283
## % of predictable range [ (model-null)/(1-null) ]:  0.04046702
## ***************************
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos     log_freq
##     3.78920      0.01539     -0.28534      0.19595
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4394 Residual
## Null Deviance:       2237
## Residual Deviance: 2077  AIC: 2085
## log likelihood:  -1038.497
## Nagelkerke R2:  0.08956176
## % pres/err predicted correctly:  -550.1814
## % of predictable range [ (model-null)/(1-null) ]:  0.04072256
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##  (Intercept)          pos      log_freq  pos:log_freq
##     3.876606     -0.277837      0.176860      0.003371
```

```
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4394 Residual
## Null Deviance:       2237
## Residual Deviance: 2077  AIC: 2085
## log likelihood:  -1038.527
## Nagelkerke R2:  0.08952878
## % pres/err predicted correctly:  -550.0281
## % of predictable range [ (model-null)/(1-null) ]:  0.04098952
## ***************************
## model index:  11
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq          I(pos^2)               pos  stimlen:log
##         4.346500          0.001542          0.123262          0.023784         -0.514405          0.
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4392 Residual
## Null Deviance:       2237
## Residual Deviance: 2073  AIC: 2085
## log likelihood:  -1036.616
## Nagelkerke R2:  0.09163032
## % pres/err predicted correctly:  -550.1378
## % of predictable range [ (model-null)/(1-null) ]:  0.04079856
## ***************************
## model index:  9
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##       (Intercept)           I(pos^2)                pos           log_freq  I(pos^2):log_freq       I
##          4.349538           0.024567          -0.517830           0.188037           0.001108
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4392 Residual
## Null Deviance:       2237
## Residual Deviance: 2073  AIC: 2085
## log likelihood:  -1036.621
## Nagelkerke R2:  0.09162412
## % pres/err predicted correctly:  -550.138
## % of predictable range [ (model-null)/(1-null) ]:  0.04079811
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq               pos  stimlen:log_freq
##         3.780115          0.017160          0.144269         -0.285507          0.006365
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4393 Residual
## Null Deviance:       2237
```

```
## Residual Deviance: 2077  AIC: 2087
## log likelihood:  -1038.449
## Nagelkerke R2:  0.08961494
## % pres/err predicted correctly:  -550.0386
## % of predictable range [ (model-null)/(1-null) ]:  0.0409712
## ***************************
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##  (Intercept)       stimlen           pos      log_freq  pos:log_freq
##     3.788449      0.014210     -0.283074      0.182154      0.002758
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4393 Residual
## Null Deviance:       2237
## Residual Deviance: 2077  AIC: 2087
## log likelihood:  -1038.48
## Nagelkerke R2:  0.08958102
## % pres/err predicted correctly:  -550.0793
## % of predictable range [ (model-null)/(1-null) ]:  0.04090036
## ***************************
## model index:  12
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##       (Intercept)            stimlen           I(pos^2)               pos           log_freq  I(pos
##          4.374965          -0.003523           0.024752          -0.518341           0.186799
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4391 Residual
## Null Deviance:       2237
## Residual Deviance: 2073  AIC: 2087
## log likelihood:  -1036.619
## Nagelkerke R2:  0.09162721
## % pres/err predicted correctly:  -550.125
## % of predictable range [ (model-null)/(1-null) ]:  0.04082086
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq               pos  stimlen:log_freq      log_fr
##        3.7805963         0.0167822         0.1446864        -0.2850039         0.0059493         0.0
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4392 Residual
## Null Deviance:       2237
## Residual Deviance: 2077  AIC: 2089
## log likelihood:  -1038.448
## Nagelkerke R2:  0.0896156
```

```
## % pres/err predicted correctly:  -550.026
## % of predictable range [ (model-null)/(1-null) ]:  0.04099317
## **************************
## model index:  10
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##       (Intercept)            stimlen           log_freq           I(pos^2)                pos   stim
##         4.3649520         -0.0010309          0.1436581          0.0245922         -0.5187774
##      log_freq:pos
##        -0.0044280
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4390 Residual
## Null Deviance:       2237
## Residual Deviance: 2073  AIC: 2089
## log likelihood:  -1036.588
## Nagelkerke R2:  0.09166059
## % pres/err predicted correctly:  -550.0688
## % of predictable range [ (model-null)/(1-null) ]:  0.04091867
## **************************
## model index:  19
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     4.33269      0.02188     -0.51415
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       2237
## Residual Deviance: 2110  AIC: 2116
## log likelihood:  -1055.033
## Nagelkerke R2:  0.07130458
## % pres/err predicted correctly:  -556.323
## % of predictable range [ (model-null)/(1-null) ]:  0.03003384
## **************************
## model index:  20
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
##     4.76423     -0.05990      0.02482     -0.52100
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4394 Residual
## Null Deviance:       2237
## Residual Deviance: 2108  AIC: 2116
## log likelihood:  -1054.141
## Nagelkerke R2:  0.07229372
## % pres/err predicted correctly:  -555.986
```

```
## % of predictable range [ (model-null)/(1-null) ]:  0.03062034
## ***************************
## model index:  16
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      3.8999      -0.2978
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       2237
## Residual Deviance: 2113  AIC: 2117
## log likelihood:  -1056.663
## Nagelkerke R2:  0.06949819
## % pres/err predicted correctly:  -556.1426
## % of predictable range [ (model-null)/(1-null) ]:  0.03034777
## ***************************
## model index:  21
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          I(pos^2)               pos  stimlen:I(pos^2)       stiml
##         3.034795          0.172878         -0.034641          0.213337          0.008023         -0.
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4392 Residual
## Null Deviance:       2237
## Residual Deviance: 2106  AIC: 2118
## log likelihood:  -1053.005
## Nagelkerke R2:  0.07355191
## % pres/err predicted correctly:  -555.5989
## % of predictable range [ (model-null)/(1-null) ]:  0.03129398
## ***************************
## model index:  17
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##     4.17076     -0.04346     -0.28168
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       2237
## Residual Deviance: 2112  AIC: 2118
## log likelihood:  -1056.176
## Nagelkerke R2:  0.07003799
## % pres/err predicted correctly:  -555.8985
## % of predictable range [ (model-null)/(1-null) ]:  0.0307726
## ***************************
## model index:  18
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##    4.411723    -0.072030    -0.344848     0.007248
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4394 Residual
## Null Deviance:       2237
## Residual Deviance: 2112  AIC: 2120
## log likelihood:  -1056.104
## Nagelkerke R2:  0.07011737
## % pres/err predicted correctly:  -555.9702
## % of predictable range [ (model-null)/(1-null) ]:  0.03064785
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     log_freq
##      3.9139      -0.1602       0.1895
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       2237
## Residual Deviance: 2168  AIC: 2174
## log likelihood:  -1083.985
## Nagelkerke R2:  0.03900725
## % pres/err predicted correctly:  -564.51
## % of predictable range [ (model-null)/(1-null) ]:  0.01578503
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq  stimlen:log_freq
##         3.907238         -0.158961          0.153034          0.004481
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4394 Residual
## Null Deviance:       2237
## Residual Deviance: 2168  AIC: 2176
## log likelihood:  -1083.96
## Nagelkerke R2:  0.03903478
## % pres/err predicted correctly:  -564.4361
## % of predictable range [ (model-null)/(1-null) ]:  0.01591373
## ***************************
## model index:  15
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
```

```
##
## Coefficients:
## (Intercept)      stimlen
##      4.2860      -0.2154
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       2237
## Residual Deviance: 2202  AIC: 2206
## log likelihood:  -1101.005
## Nagelkerke R2:  0.01982059
## % pres/err predicted correctly:  -569.0766
## % of predictable range [ (model-null)/(1-null) ]:  0.00783731
## **************************
## model index:  14
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       2.583
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4397 Residual
## Null Deviance:       2237
## Residual Deviance: 2237  AIC: 2239
## log likelihood:  -1118.451
## Nagelkerke R2:  -5.56958e-16
## % pres/err predicted correctly:  -573.5798
## % of predictable range [ (model-null)/(1-null) ]:  0
## **************************
```

```r
BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                         AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2

FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                       by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                               RunLabel,"_",CurTask,
                               "_freq_len_pos_model_summary.csv"),row.names = FALSE)
kable(FLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	log_freq	pos	stimlen:log_freq	log_freq:pos	stimlen:pos	I(pos^2)	log_freq:I(pos^2)	stimlen:I(pos^2)	stimlen:log_freq:pos
preserved ~ pos + log_freq	2083.1.08 0.000 1.0000 0.2600185294986599NA							0.193489 NA3	-0.2801150	NA	NA	NA	NA	NA	NA	NA

Model	AIC DeltaAIC expAIC AICw NagR2 (Intercept)	stimlen	log_freq	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	stimlen:I(pos^2)
preserved ~ stimlen + I(pos^2) + pos + log_freq	2083.[illegible]	-0.0007470	0.1944093 / NA	-0.5111340	NA	NA	0.0234694 / NA		NA	NA	NA
preserved ~ stimlen + pos + log_freq	2084.[illegible]	[illegible]	NA	-0.2853429	NA	NA	NA	NA	NA	NA	NA
preserved ~ pos * log_freq	2085.[illegible]	NA	0.1768601 / NA	-0.2778370	0.0033709 / NA		NA	NA	NA	NA	NA
preserved ~ stimlen * log_freq + I(pos^2) + pos	2085.[illegible]	[illegible]	[illegible]	-0.5144054	NA	NA	0.0237835 / NA		NA	NA	NA
preserved ~ (I(pos^2) + pos) * log_freq	2085.[illegible]	NA	0.1880370 / NA	-0.517[illegible]	-[illegible]651650	NA	0.0245[illegible] / NA			NA	NA
preserved ~ stimlen * log_freq + pos	2086.[illegible]	[illegible]	[illegible]	-0.2855072	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen + pos * log_freq	2086.[illegible]	[illegible]	NA	-0.2830741	0.0027583 / NA		NA	NA	NA	NA	NA
preserved ~ stimlen + (I(pos^2) + pos) * log_freq	2087.[illegible]	-0.0035230	0.1867901 / NA	-0.518[illegible]	-[illegible]0550776	NA	0.0240[illegible] / NA			NA	NA
preserved ~ stimlen * log_freq + pos * log_freq	2088.[illegible]	[illegible]	[illegible]	-0.2850039	NA	0.0005911 / NA		NA	NA	NA	NA
preserved ~ stimlen * log_freq + (I(pos^2) + pos) * log_freq	2089.[illegible]	-0.0010309	0.1436[illegible]	-0.5187774	NA	-0.0044280	0.0245922 / NA		0.0008311 / NA		NA
preserved ~ I(pos^2) + pos	2116.[illegible]	NA	NA	-0.5141548	NA	NA	0.0218799 / NA		NA	NA	NA
preserved ~ stimlen + I(pos^2) + pos	2116.[illegible]	-0.0599038	NA	-0.5209953	NA	NA	0.0248159 / NA		NA	NA	NA

Model	AIC	DeltaAIC	AICexp	AICw	NagR2	(Intercept)	stimlen	log_freq	stimlen:log_freq	pos	log_freq:pos	[illegible]	I(pos^2)	[illegible]	[illegible]	stimlen:pos	stimlen:I(pos^2)
preserved ~ pos	2117.[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	NA	NA	NA	-0.2977816	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen * (I(pos^2) + pos)	2118.[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	NA	NA	0.2133367	[illegible]	NA	-0.0346408	NA	NA	-0.0979311	0.0080234
preserved ~ stimlen + pos	2118.[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	-0.0434578	NA	NA	-0.2816778	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen * pos	2120.[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	-0.0720303	NA	NA	-0.3448481	NA	NA	NA	NA	NA	0.0072483	NA
preserved ~ stimlen + log_freq	2173.[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	-0.1601755	0.1895235	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen * log_freq	2175.[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	-0.1589611	0.1530[illegible]	0.0424806	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen	2206.[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	-0.2154392	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ 1	2238.[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA

```r
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ pos + log_freq"
```

```r
print(BestFLPModel)
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos     log_freq
##      3.8866      -0.2801       0.1935
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       2237
## Residual Deviance: 2077  AIC: 2083
```

```r
# do a median split on frequency to plot hf/ls effects (analysis is continuous)
median_freq <- median(PosDat$log_freq)
PosDat$freq_bin[PosDat$log_freq >= median_freq]<-"hf"
PosDat$freq_bin[PosDat$log_freq <  median_freq]<-"lf"
```

```r
PosDat$FLPFitted<-fitted(BestFLPModel)

HFDat <- PosDat[PosDat$freq_bin == "hf",]
LFDat <- PosDat[PosDat$freq_bin == "lf",]

HF_Plot <- plot_len_pos_obs_predicted(HFDat,paste0(CurPat," - ",RunLabel," - High frequency"),"FLPFitte
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

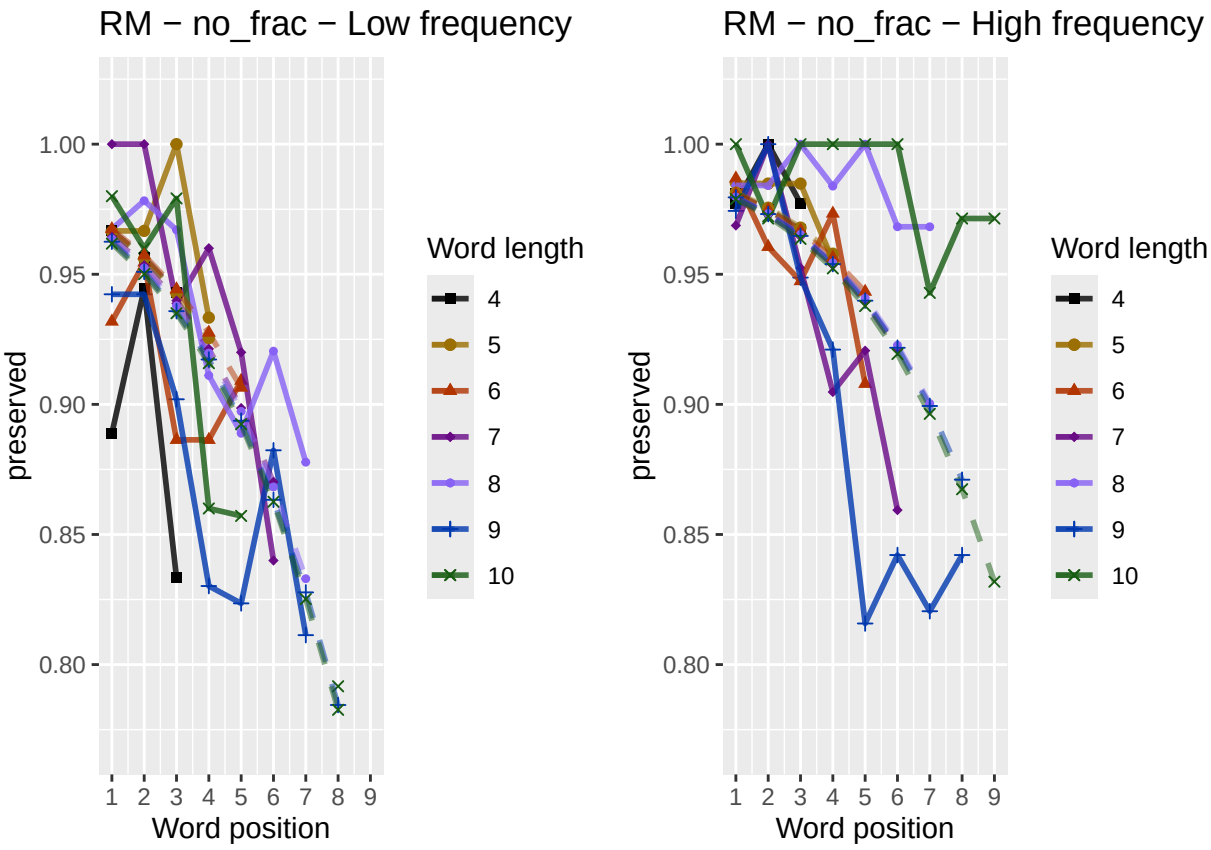
```
LF_Plot <- plot_len_pos_obs_predicted(LFDat,paste0(CurPat," - ",RunLabel," - Low frequency"),"FLPFitted
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
library(ggpubr)
Both_Plots <- ggarrange(LF_Plot,HF_Plot) # labels=c("LF","HF",ncol=2)
```

```
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: Removed 2 rows containing missing values or values outside the scale range (`geom_line()`).
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_line()`).
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_point()`).
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",
              CurTask,"_frequency_effect_length_pos_wfit.png"),
       device="png",unit="cm",width=30,height=11)
print(Both_Plots)
```




```
# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
```

```
  "preserved ~ stimlen",
  "preserved ~ 1"
)
MERes<-TestModels(MEModelEquations,PosDat)
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.390       -2.459
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       2237
## Residual Deviance: 1442  AIC: 1446
## log likelihood:  -721.1634
## Nagelkerke R2:  0.4145919
## % pres/err predicted correctly:  -345.2065
## % of predictable range [ (model-null)/(1-null) ]:  0.3974615
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     4.33269      0.02188     -0.51415
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       2237
## Residual Deviance: 2110  AIC: 2116
## log likelihood:  -1055.033
## Nagelkerke R2:  0.07130458
## % pres/err predicted correctly:  -556.323
## % of predictable range [ (model-null)/(1-null) ]:  0.03003384
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      3.8999      -0.2978
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       2237
## Residual Deviance: 2113  AIC: 2117
## log likelihood:  -1056.663
## Nagelkerke R2:  0.06949819
```

```
## % pres/err predicted correctly:  -556.1426
## % of predictable range [ (model-null)/(1-null) ]:  0.03034777
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.2860      -0.2154
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       2237
## Residual Deviance: 2202  AIC: 2206
## log likelihood:  -1101.005
## Nagelkerke R2:  0.01982059
## % pres/err predicted correctly:  -569.0766
## % of predictable range [ (model-null)/(1-null) ]:  0.00783731
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##     2.47333      0.04225
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       2237
## Residual Deviance: 2235  AIC: 2239
## log likelihood:  -1117.385
## Nagelkerke R2:  0.001214959
## % pres/err predicted correctly:  -573.3342
## % of predictable range [ (model-null)/(1-null) ]:  0.0004273757
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       2.583
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4397 Residual
## Null Deviance:       2237
## Residual Deviance: 2237  AIC: 2239
## log likelihood:  -1118.451
## Nagelkerke R2:  -5.56958e-16
## % pres/err predicted correctly:  -573.5798
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
```

```
BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

MEAICSummary<-data.frame(Model=MERes$Model,
                         AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                            by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                              CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1446.327	0.0000	1	1	0.4145919	3.389948	NA	-2.459125	NA	NA	NA
preserved ~ (I(pos^2) + pos)	2116.067	669.7400	0	0	0.0713046	4.332694	NA	NA	0.0218799	-0.5141548	NA
preserved ~ pos	2117.325	670.9987	0	0	0.0694983	2.899862	NA	NA	NA	-0.2977816	NA
preserved ~ stimlen	2206.010	759.6838	0	0	0.0198206	4.286008	NA	NA	NA	NA	-0.2154392
preserved ~ CumPres	2238.771	792.4438	0	0	0.0012150	2.473327	0.0422457	NA	NA	NA	NA
preserved ~ 1	2238.901	792.5746	0	0	0.0000000	2.582714	NA	NA	NA	NA	NA

```
if(DoSimulations){
      BestMEModelFormulaRnd <- BestMEModelFormula
    if(grepl("CumPres",BestMEModelFormulaRnd)){
      BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
    }else if(grepl("CumErr",BestMEModelFormulaRnd)){
      BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
    }

    RndModelAIC<-numeric(length=RandomSamples)
    for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
      PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
      PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
      BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                          family="binomial",data=PosDat)
      RndModelAIC[rindex] <- BestModelRnd$aic
    }
    ModelNames<-c(paste0("***",BestMEModelFormula),
                  rep(BestMEModelFormulaRnd,RandomSamples))
    AICValues <- c(BestMEModel$aic,RndModelAIC)
```

```
    BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
    BestMEModelRndDF <- BestMEModelRndDF %>% arrange(AIC)
    BestMEModelRndDF <- rbind(BestMEModelRndDF,
                              data.frame(Name=c("Random average"),
                                         AIC=c(mean(RndModelAIC))))
    BestMEModelRndDF <- rbind(BestMEModelRndDF,
                              data.frame(Name=c("Random SD"),
                                         AIC=c(sd(RndModelAIC))))

    write.csv(BestMEModelRndDF,
              paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                     "_best_main_effects_model_with_random_cum_term.csv"),
              row.names = FALSE)
}
```

```
syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary,paste0(TablesDir,CurPat,"_",
                                        RunLabel,"_",CurTask,
                                        "_syllable_component_summary.csv"),row.names = FALSE)
kable(syll_component_summary)
```

syll_component	MeanPres	N
1	0.9288194	576
O	0.9189587	2036
P	0.9459459	37
S	0.9143969	257
V	0.9470509	1492

```
#  main effects models for data without satellite positions

keep_components = c("O","V","1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                              stimlen,stim,pos,
                              preserved,syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data,MEModelEquations)
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.388       -2.626
##
## Degrees of Freedom: 4103 Total (i.e. Null);  4102 Residual
```

```
## Null Deviance:       2070
## Residual Deviance: 1326  AIC: 1330
## log likelihood:  -662.8302
## Nagelkerke R2:  0.4187589
## % pres/err predicted correctly:  -313.8967
## % of predictable range [ (model-null)/(1-null) ]:  0.406322
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     4.34545      0.02422     -0.52639
##
## Degrees of Freedom: 4103 Total (i.e. Null);  4101 Residual
## Null Deviance:       2070
## Residual Deviance: 1960  AIC: 1966
## log likelihood:  -980.1438
## Nagelkerke R2:  0.0666233
## % pres/err predicted correctly:  -514.8839
## % of predictable range [ (model-null)/(1-null) ]:  0.02739875
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      3.8680      -0.2874
##
## Degrees of Freedom: 4103 Total (i.e. Null);  4102 Residual
## Null Deviance:       2070
## Residual Deviance: 1964  AIC: 1968
## log likelihood:  -981.9739
## Nagelkerke R2:  0.06443037
## % pres/err predicted correctly:  -514.7912
## % of predictable range [ (model-null)/(1-null) ]:  0.02757348
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.2995      -0.2154
##
## Degrees of Freedom: 4103 Total (i.e. Null);  4102 Residual
## Null Deviance:       2070
## Residual Deviance: 2038  AIC: 2042
## log likelihood:  -1018.916
```

```
## Nagelkerke R2:  0.01974269
## % pres/err predicted correctly:  -525.2969
## % of predictable range [ (model-null)/(1-null) ]:  0.007767019
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##     2.38251      0.09094
##
## Degrees of Freedom: 4103 Total (i.e. Null);  4102 Residual
## Null Deviance:       2070
## Residual Deviance: 2062  AIC: 2066
## log likelihood:  -1030.983
## Nagelkerke R2:  0.004969819
## % pres/err predicted correctly:  -528.4825
## % of predictable range [ (model-null)/(1-null) ]:  0.001761114
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       2.595
##
## Degrees of Freedom: 4103 Total (i.e. Null);  4103 Residual
## Null Deviance:       2070
## Residual Deviance: 2070  AIC: 2072
## log likelihood:  -1035.027
## Nagelkerke R2:  2.802662e-16
## % pres/err predicted correctly:  -529.4167
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
```

```
write.csv(SimpSyllMEAICSummary,
          paste0(TablesDir,CurPat,"_",RunLabel,"_",
                 CurTask,
                 "_OV1_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(posˆ2)	pos	stimlen
preserved ~ CumErr	1329.660	0.0000	1	1	0.4187589	3.387541	NA	-2.62555	NA	NA	NA
preserved ~ (I(posˆ2) + pos)	1966.288	636.6272	0	0	0.0666234	1.345451	NA	NA	0.0242195	-0.5263924	NA
preserved ~ pos	1967.948	638.2873	0	0	0.0644303	1.868000	NA	NA	NA	-0.2873938	NA

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ stimlen	2041.831	712.1709	0	0	0.0197427	4.299477	NA	NA	NA	NA	-0.2154161
preserved ~ CumPres	2065.966	736.3057	0	0	0.0049698	3.382513	0.0909429	NA	NA	NA	NA
preserved ~ 1	2072.054	742.3933	0	0	0.0000000	2.595255	NA	NA	NA	NA	NA

```
#  main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
#  also reduces data)

keep_components = c("O","V")
OVData <- PosDat[PosDat$syll_component %in% keep_components,]
OVData <- OVData %>% select(stim_number,
                            stimlen,stim,pos,
                            preserved,syll_component)
OVData$CumPres <- CalcCumPres(OVData)
OVData$CumErr <- CalcCumErrFromPreserved(OVData)

SimpSyllMEAICSummary2<-EvaluateSubsetData(OVData,MEModelEquations)
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.293       -2.800
##
## Degrees of Freedom: 3527 Total (i.e. Null);  3526 Residual
## Null Deviance:       1774
## Residual Deviance: 1209  AIC: 1213
## log likelihood:  -604.277
## Nagelkerke R2:  0.3748793
## % pres/err predicted correctly:  -287.9963
## % of predictable range [ (model-null)/(1-null) ]:  0.3637939
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     4.21936      0.02272     -0.49183
##
## Degrees of Freedom: 3527 Total (i.e. Null);  3525 Residual
## Null Deviance:       1774
## Residual Deviance: 1687  AIC: 1693
## log likelihood:  -843.2999
## Nagelkerke R2:  0.06213615
```

```
## % pres/err predicted correctly:  -441.8242
## % of predictable range [ (model-null)/(1-null) ]:  0.0251518
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##       3.789       -0.270
##
## Degrees of Freedom: 3527 Total (i.e. Null);  3526 Residual
## Null Deviance:       1774
## Residual Deviance: 1690  AIC: 1694
## log likelihood:  -844.8197
## Nagelkerke R2:  0.0600089
## % pres/err predicted correctly:  -441.7044
## % of predictable range [ (model-null)/(1-null) ]:  0.02541554
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.0969      -0.1903
##
## Degrees of Freedom: 3527 Total (i.e. Null);  3526 Residual
## Null Deviance:       1774
## Residual Deviance: 1752  AIC: 1756
## log likelihood:  -876.0925
## Nagelkerke R2:  0.01582792
## % pres/err predicted correctly:  -450.4007
## % of predictable range [ (model-null)/(1-null) ]:  0.006271396
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##         2.6
##
## Degrees of Freedom: 3527 Total (i.e. Null);  3527 Residual
## Null Deviance:       1774
## Residual Deviance: 1774  AIC: 1776
## log likelihood:  -887.1625
## Nagelkerke R2:  5.617943e-16
## % pres/err predicted correctly:  -453.2494
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
```

```
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##     2.48961      0.05433
##
## Degrees of Freedom: 3527 Total (i.e. Null);  3526 Residual
## Null Deviance:       1774
## Residual Deviance: 1772  AIC: 1776
## log likelihood:  -886.1908
## Nagelkerke R2:  0.001393269
## % pres/err predicted correctly:  -453.026
## % of predictable range [ (model-null)/(1-null) ]:  0.00049179
## ***************************
```

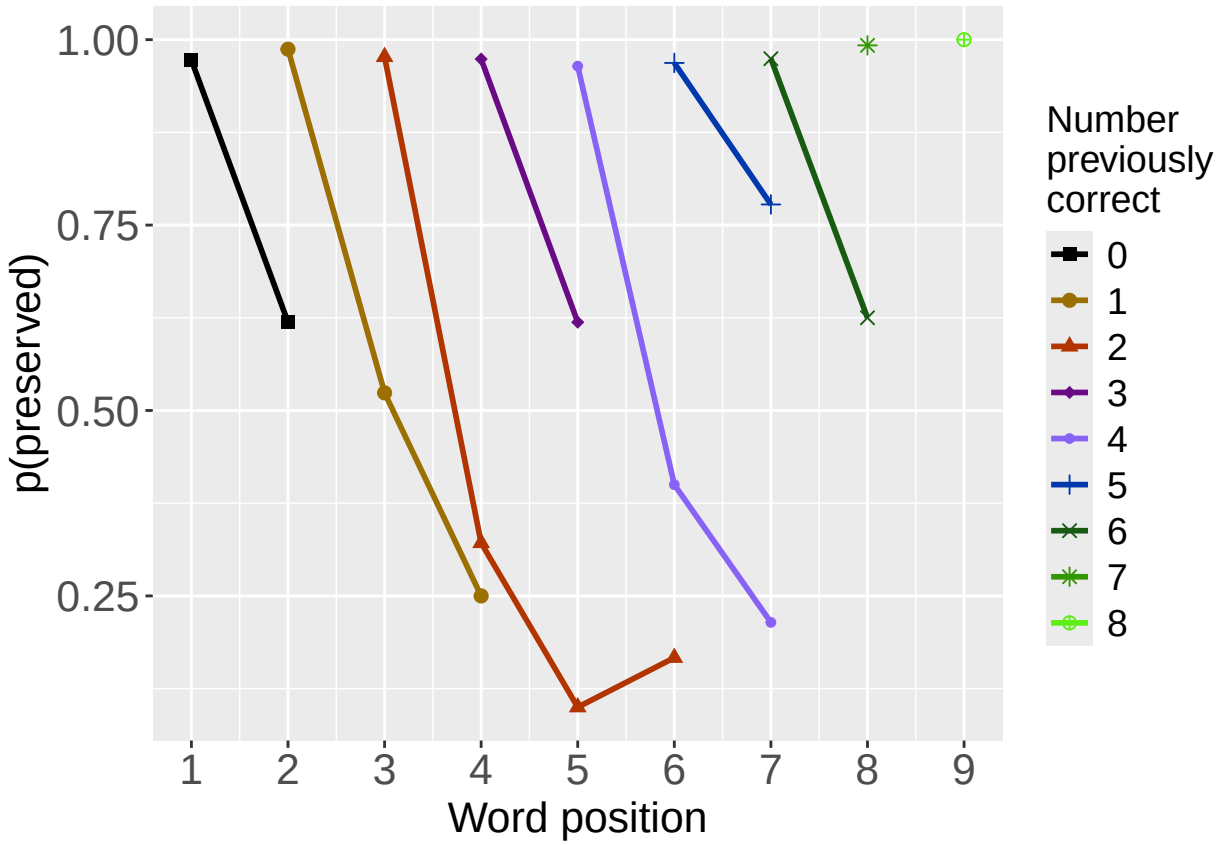
```
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                 "_OV_main_effects_model_summary.csv"),row.names = FALSE)
kable(SimpSyllMEAICSummary2)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1212.554	0.0000	1	1	0.374879	3.293139	NA	-2.799748	NA	NA	NA
preserved ~ (I(pos^2) + pos)	1692.600	480.0459	0	0	0.062136	2.219357	NA	NA	0.0227157	-0.4918324	NA
preserved ~ pos	1693.639	481.0855	0	0	0.060008	3.788645	NA	NA	NA	-0.2699709	NA
preserved ~ stimlen	1756.185	543.6310	0	0	0.015827	4.096932	NA	NA	NA	NA	-0.1903206
preserved ~ 1	1776.325	563.7710	0	0	0.000000	2.599649	NA	NA	NA	NA	NA
preserved ~ CumPres	1776.382	563.8277	0	0	0.001393	2.489615	0.0543281	NA	NA	NA	NA

```
# plot prev err and prev cor plots
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

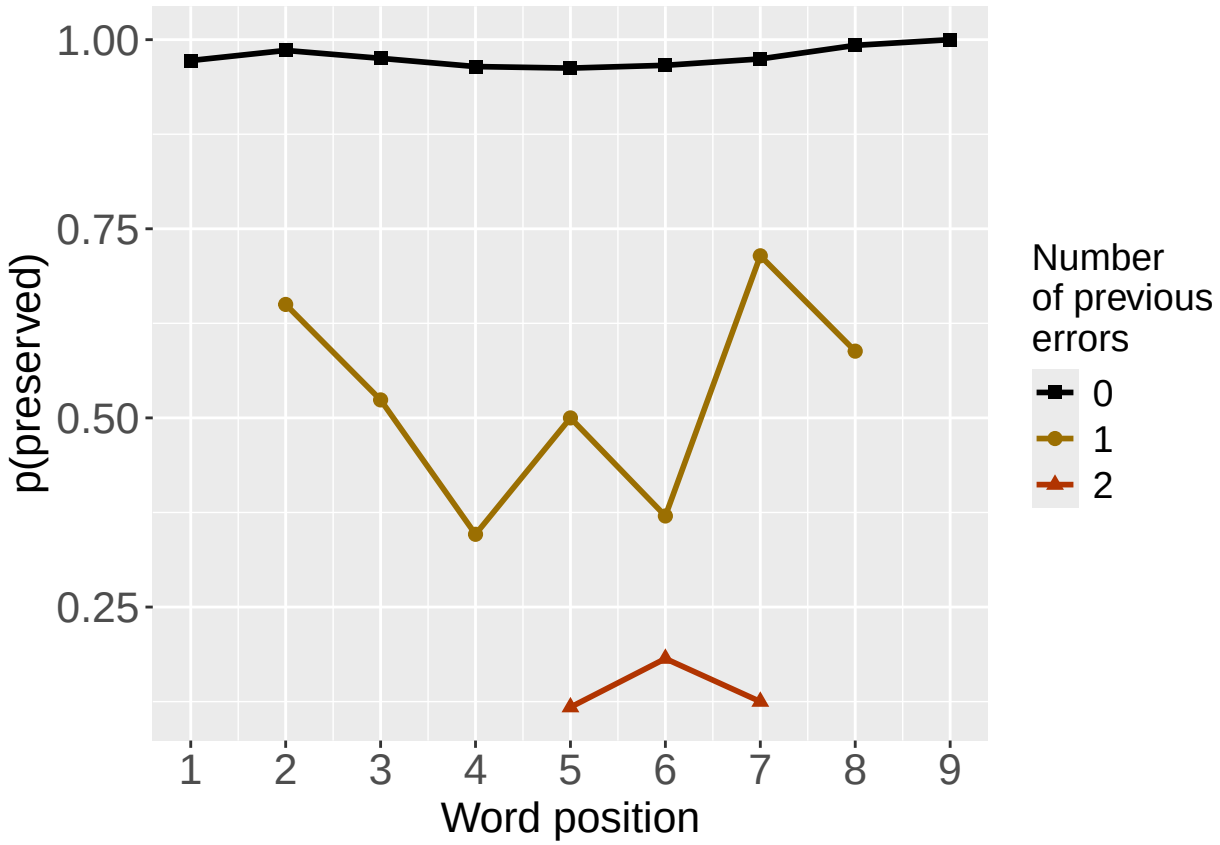
```
print(PrevCorPlot)
```




```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```




```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

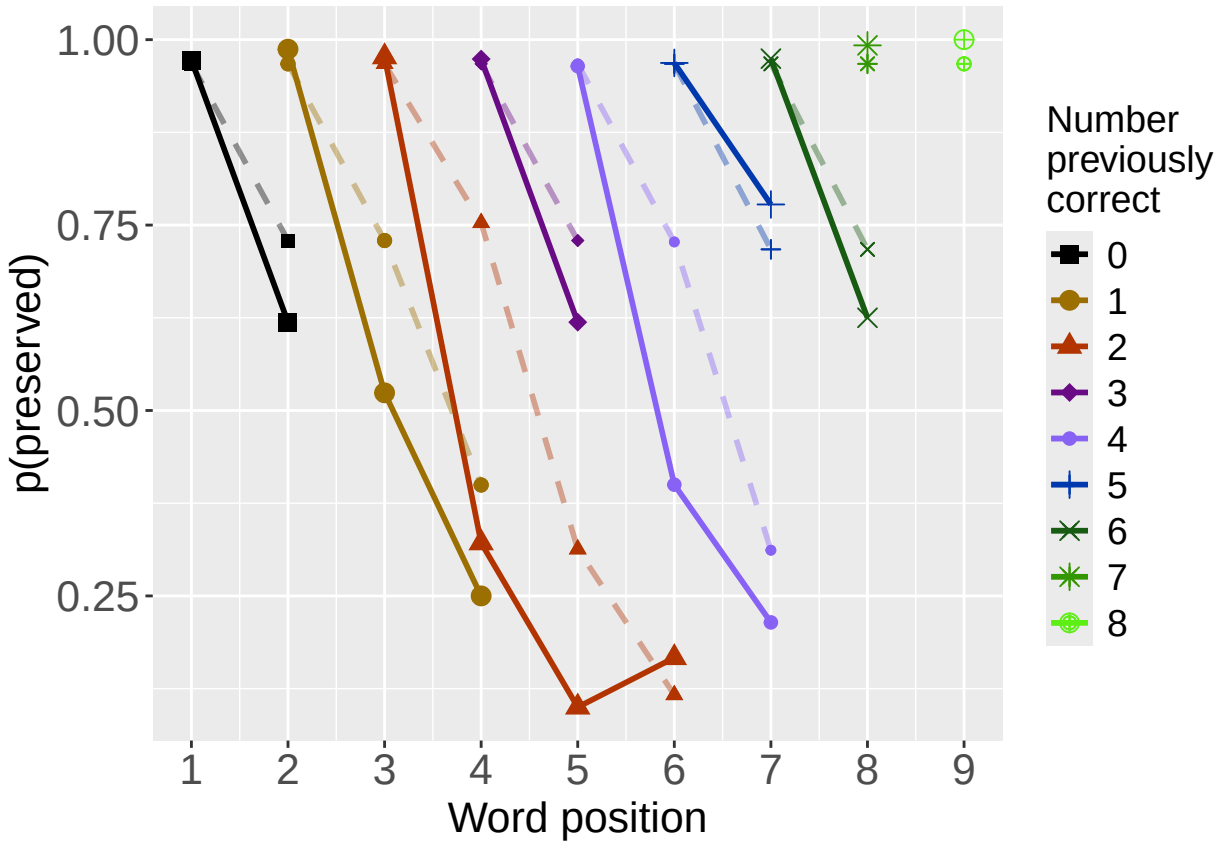
```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

```
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

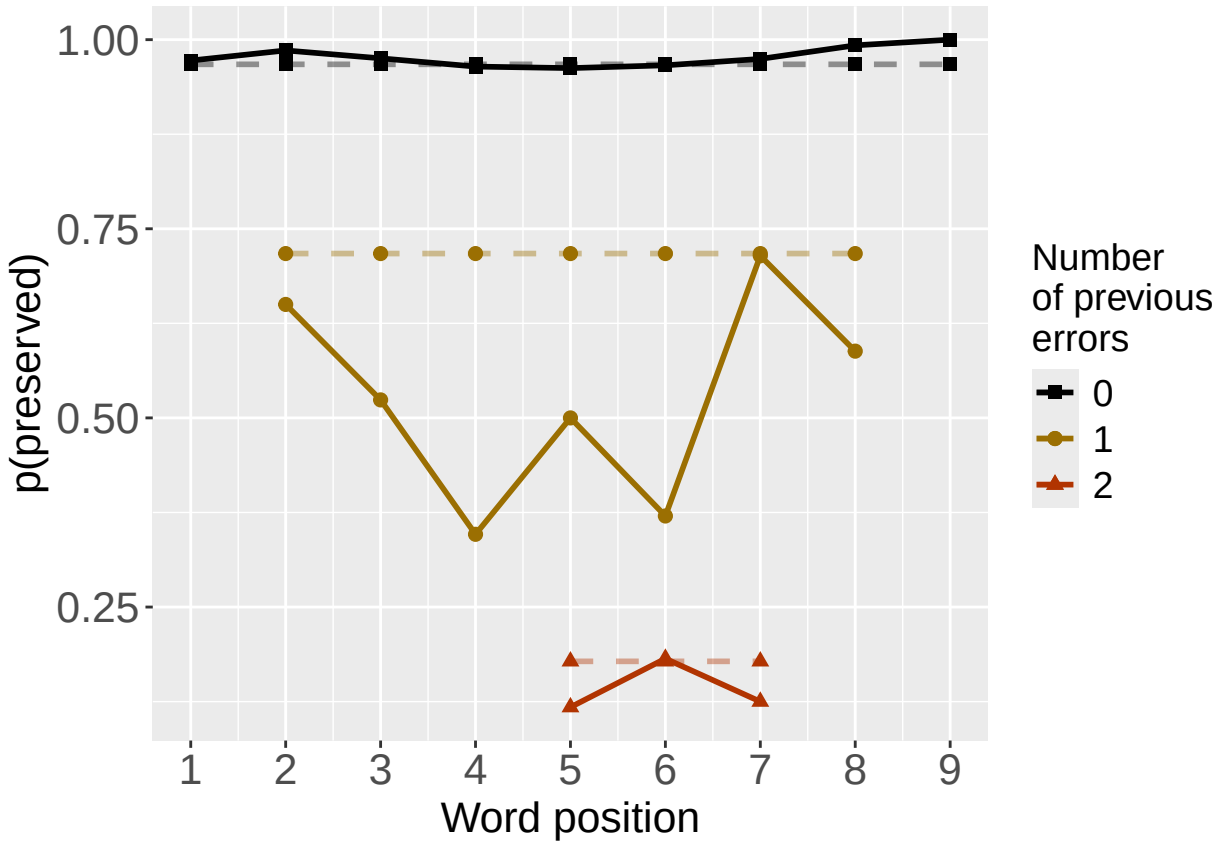
```
print(PrevCorPlot)
```




```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```




```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                 CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                 CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

```
CumAICSummary <- NULL
```

```
########
# level 2 -- Add position squared (quadratic with position)
########

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr     I(pos^2)          pos
##     4.30440     -2.41874      0.04848     -0.47495
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4394 Residual
## Null Deviance:       2237
## Residual Deviance: 1432  AIC: 1440
## log likelihood:  -716.235
## Nagelkerke R2:  0.419279
## % pres/err predicted correctly:  -343.4862
## % of predictable range [ (model-null)/(1-null) ]:  0.4004555
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.390       -2.459
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       2237
## Residual Deviance: 1442  AIC: 1446
## log likelihood:  -721.1634
## Nagelkerke R2:  0.4145919
## % pres/err predicted correctly:  -345.2065
## % of predictable range [ (model-null)/(1-null) ]:  0.3974615
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     4.33269      0.02188     -0.51415
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       2237
## Residual Deviance: 2110  AIC: 2116
## log likelihood:  -1055.033
## Nagelkerke R2:  0.07130458
## % pres/err predicted correctly:  -556.323
## % of predictable range [ (model-null)/(1-null) ]:  0.03003384
## ***************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos
preserved ~ CumErr + I(pos^2) + pos	1440.470	0.000000	1.0000000	0.9492284	0.4192790	4.304399	-2.418740	0.0484763	-0.4749470
preserved ~ CumErr	1446.327	5.856625	0.0534872	0.0507716	0.4145919	3.389948	-2.459125	NA	NA
preserved ~ I(pos^2) + pos	2116.067	675.596619	0.0000000	0.0000000	0.0713046	4.332694	NA	0.0218799	-0.5141548

```
########
# level 2 -- Add length
########

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## ****************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.390       -2.459
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       2237
## Residual Deviance: 1442  AIC: 1446
## log likelihood:  -721.1634
## Nagelkerke R2:  0.4145919
## % pres/err predicted correctly:  -345.2065
## % of predictable range [ (model-null)/(1-null) ]:  0.3974615
## ****************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      stimlen
##      3.8019      -2.4391      -0.0536
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       2237
## Residual Deviance: 1441  AIC: 1447
## log likelihood:  -720.4625
## Nagelkerke R2:  0.4152591
## % pres/err predicted correctly:  -345.3311
## % of predictable range [ (model-null)/(1-null) ]:  0.3972446
## ****************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##      4.2860      -0.2154
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       2237
## Residual Deviance: 2202  AIC: 2206
## log likelihood:  -1101.005
## Nagelkerke R2:  0.01982059
## % pres/err predicted correctly:  -569.0766
## % of predictable range [ (model-null)/(1-null) ]:  0.00783731
## ***************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen
preserved ~ CumErr	1446.327	0.0000000	1.0000000	0.5742256	0.4145919	3.389948	-2.459125	NA
preserved ~ CumErr + stimlen	1446.925	0.5982251	0.7414759	0.4257744	0.4152591	3.801862	-2.439080	-0.0535970
preserved ~ stimlen	2206.010	759.68376370	0.0000000	0.0000000	0.0198206	4.286008	NA	-0.2154392

```
########
# level 2 -- add cumulative preserved
########

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.390       -2.459
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       2237
## Residual Deviance: 1442  AIC: 1446
## log likelihood:  -721.1634
## Nagelkerke R2:  0.4145919
## % pres/err predicted correctly:  -345.2065
## % of predictable range [ (model-null)/(1-null) ]:  0.3974615
## ***************************
## model index:  2
##
```

```
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      CumPres
##    3.386620    -2.459192     0.001235
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       2237
## Residual Deviance: 1442  AIC: 1448
## log likelihood:  -721.1628
## Nagelkerke R2:  0.4145924
## % pres/err predicted correctly:  -345.1991
## % of predictable range [ (model-null)/(1-null) ]:  0.3974743
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##     2.47333      0.04225
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       2237
## Residual Deviance: 2235  AIC: 2239
## log likelihood:  -1117.385
## Nagelkerke R2:  0.001214959
## % pres/err predicted correctly:  -573.3342
## % of predictable range [ (model-null)/(1-null) ]:  0.0004273757
## ***************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres
preserved ~ CumErr	1446.327	0.000000	1.0000000	0.7309501	0.4145919	3.389948	-2.459125	NA
preserved ~ CumErr + CumPres	1448.326	1.998897	0.3680824	0.2690499	0.4145924	3.386620	-2.459192	0.0012345
preserved ~ CumPres	2238.771	792.443804	0.0000000	0.0000000	0.0012150	2.473327	NA	0.0422457

```
########
# level 2 -- Add linear position (NOT quadratic)
########

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.390       -2.459
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       2237
## Residual Deviance: 1442  AIC: 1446
## log likelihood:  -721.1634
## Nagelkerke R2:  0.4145919
## % pres/err predicted correctly:  -345.2065
## % of predictable range [ (model-null)/(1-null) ]:  0.3974615
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr          pos
##     3.53307     -2.41551     -0.03784
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       2237
## Residual Deviance: 1441  AIC: 1447
## log likelihood:  -720.6356
## Nagelkerke R2:  0.4150943
## % pres/err predicted correctly:  -345.2311
## % of predictable range [ (model-null)/(1-null) ]:  0.3974187
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      3.8999      -0.2978
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       2237
## Residual Deviance: 2113  AIC: 2117
## log likelihood:  -1056.663
## Nagelkerke R2:  0.06949819
## % pres/err predicted correctly:  -556.1426
## % of predictable range [ (model-null)/(1-null) ]:  0.03034777
## ***************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	pos
preserved ~ CumErr	1446.327	0.0000000	1.0000000	0.6159195	0.4145919	3.389948	-2.459125	NA
preserved ~ CumErr + pos	1447.271	0.9445282	0.6235888	0.3840805	0.4150943	3.533070	-2.415505	-0.0378379
preserved ~ pos	2117.325	670.9987209	0.0000000	0.0000000	0.0694982	3.899862	NA	-0.2977816

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary,paste0(TablesDir,CurPat,"_",
                               RunLabel,"_",CurTask,
                               "_main_effects_plus_one_model_summary.csv"),row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	stimlen	CumPres
preserved ~ CumErr + I(pos^2) + pos	1440.470	0.0000000	1.0000000	0.9492284	0.4419279	4.0304399	-2.418740	0.0484763	-0.4749470	NA	NA
preserved ~ CumErr	1446.325	5.8566246	0.0534872	0.0507716	0.4145919	3.389948	-2.459125	NA	NA	NA	NA
preserved ~ CumErr	1446.327	7.0000000	1.0000000	0.0574225	0.4145919	3.389948	-2.459125	NA	NA	NA	NA
preserved ~ CumErr	1446.327	7.0000000	1.0000000	0.0730950	0.4145919	3.389948	-2.459125	NA	NA	NA	NA
preserved ~ CumErr	1446.327	7.0000000	1.0000000	0.0615919	0.4145919	3.389948	-2.459125	NA	NA	NA	NA
preserved ~ CumErr + stimlen	1446.926	7.5982251	0.7414759	0.4257744	0.4415259	4.1801862	-2.439080	NA	NA	-0.0535970	NA
preserved ~ CumErr + pos	1447.271	0.9445282	0.6235888	0.3840805	0.4150943	3.533070	-2.415505	NA	-0.0378379	NA	NA
preserved ~ CumErr + CumPres	1448.326	8.9988969	0.3680824	0.2269049	0.4414592	4.4386620	-2.459192	NA	NA	NA	0.0012345
preserved ~ I(pos^2) + pos	2116.065	675.5966196	0.0000000	0.0000000	0.0071304	6.6332694	NA	0.0218799	-0.5141548	NA	NA
preserved ~ pos	2117.325	670.9987209	0.0000000	0.0000000	0.0069498	3.899862	NA	NA	-0.2977816	NA	NA
preserved ~ stimlen	2206.017	759.6837687	0.0000000	0.0000000	0.0019820	6.6286008	NA	NA	NA	-0.2154392	NA
preserved ~ CumPres	2238.777	792.4438039	0.0000000	0.0000000	0.0001215	4.0473327	NA	NA	NA	NA	0.0422457

```
# explore influence of frequency and length

if(grepl("stimlen",BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
      BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
      paste0(BestModelFormulaL2," + log_freq")
  )
```

```
}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
    Level3ModelEquations<-c(
      BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
      paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
      BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
      paste0(BestModelFormulaL2," + log_freq"),
      paste0(BestModelFormulaL2," + stimlen"),
      paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)
```

```
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr     I(pos^2)          pos     log_freq
##     4.31510     -2.38121      0.05087     -0.47918      0.15653
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4393 Residual
## Null Deviance:       2237
## Residual Deviance: 1418  AIC: 1428
## log likelihood:  -708.7628
## Nagelkerke R2:  0.4263655
## % pres/err predicted correctly:  -343.5115
## % of predictable range [ (model-null)/(1-null) ]:  0.4004114
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr     I(pos^2)          pos      stimlen     log_freq
##     4.48318     -2.38281      0.05207     -0.48267     -0.02312      0.15211
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4392 Residual
## Null Deviance:       2237
## Residual Deviance: 1417  AIC: 1429
## log likelihood:  -708.6674
## Nagelkerke R2:  0.4264558
## % pres/err predicted correctly:  -343.4993
## % of predictable range [ (model-null)/(1-null) ]:  0.4004326
## ***************************
## model index:  1
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr     I(pos^2)          pos
##     4.30440     -2.41874      0.04848     -0.47495
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4394 Residual
## Null Deviance:       2237
## Residual Deviance: 1432  AIC: 1440
## log likelihood:  -716.235
## Nagelkerke R2:  0.419279
## % pres/err predicted correctly:  -343.4862
## % of predictable range [ (model-null)/(1-null) ]:  0.4004555
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr     I(pos^2)          pos      stimlen
##     4.80354     -2.41985      0.05222     -0.48482     -0.06900
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4393 Residual
## Null Deviance:       2237
## Residual Deviance: 1431  AIC: 1441
## log likelihood:  -715.32
## Nagelkerke R2:  0.420148
## % pres/err predicted correctly:  -343.5058
## % of predictable range [ (model-null)/(1-null) ]:  0.4004213
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       2.583
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4397 Residual
## Null Deviance:       2237
## Residual Deviance: 2237  AIC: 2239
## log likelihood:  -1118.451
## Nagelkerke R2:  -5.56958e-16
## % pres/err predicted correctly:  -573.5798
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
```

```
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]
```

```
AICSummary<-data.frame(Model=Level3Res$Model,
                       AIC=Level3Res$AIC,
                       row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                          by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                            CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	log_freq	stimlen
preserved ~ CumErr + I(pos^2) + pos + log_freq	1427.526	0.0000000	1.0000000	0.7104063	0.4263645	3.315096	-2.381212	0.0508727	-0.4791804	0.1565282	NA
preserved ~ CumErr + I(pos^2) + pos + stimlen + log_freq	1429.336	1.8093190	0.4046790	0.2874870	0.4264548	3.183180	-2.382812	0.0520691	-0.4826746	0.1521138	-0.0231155
preserved ~ CumErr + I(pos^2) + pos	1440.470	12.9445330	0.0015467	0.0010981	0.4192790	3.304399	-2.418740	0.0484763	-0.4749470	NA	NA
preserved ~ CumErr + I(pos^2) + pos + stimlen	1440.640	13.1145310	0.0014198	0.0010086	0.4201480	3.303540	-2.419852	0.0522219	-0.4848179	NA	-0.0689981
preserved ~ 1	2238.908	811.3757470	0.0000000	0.0000000	0.0000000	2.582714	NA	NA	NA	NA	NA

```
# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions
```

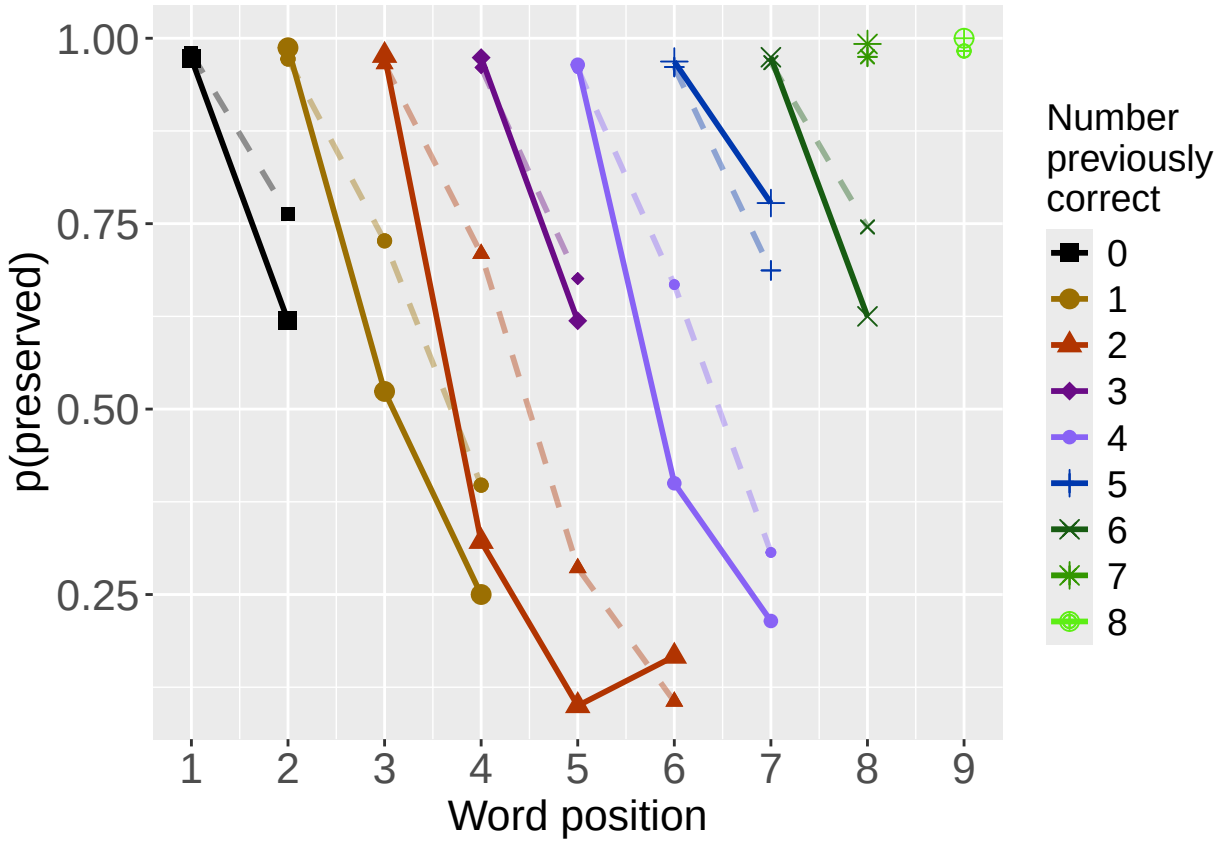
```
## Single term deletions
##
## Model:
## preserved ~ CumErr + I(pos^2) + pos + log_freq
##          Df Deviance    AIC
## CumErr    1   2073.4 2081.4
## log_freq  1   1432.5 1440.5
## pos       1   1427.5 1435.5
## I(pos^2)  1   1427.2 1435.2
## <none>        1417.5 1427.5
```

```
################################
# Single deletions from best model
################################

write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                    RunLabel,"_",CurTask,
                                    "_best_model_single_term_deletions.csv"),
          row.names = TRUE)
```

```
# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

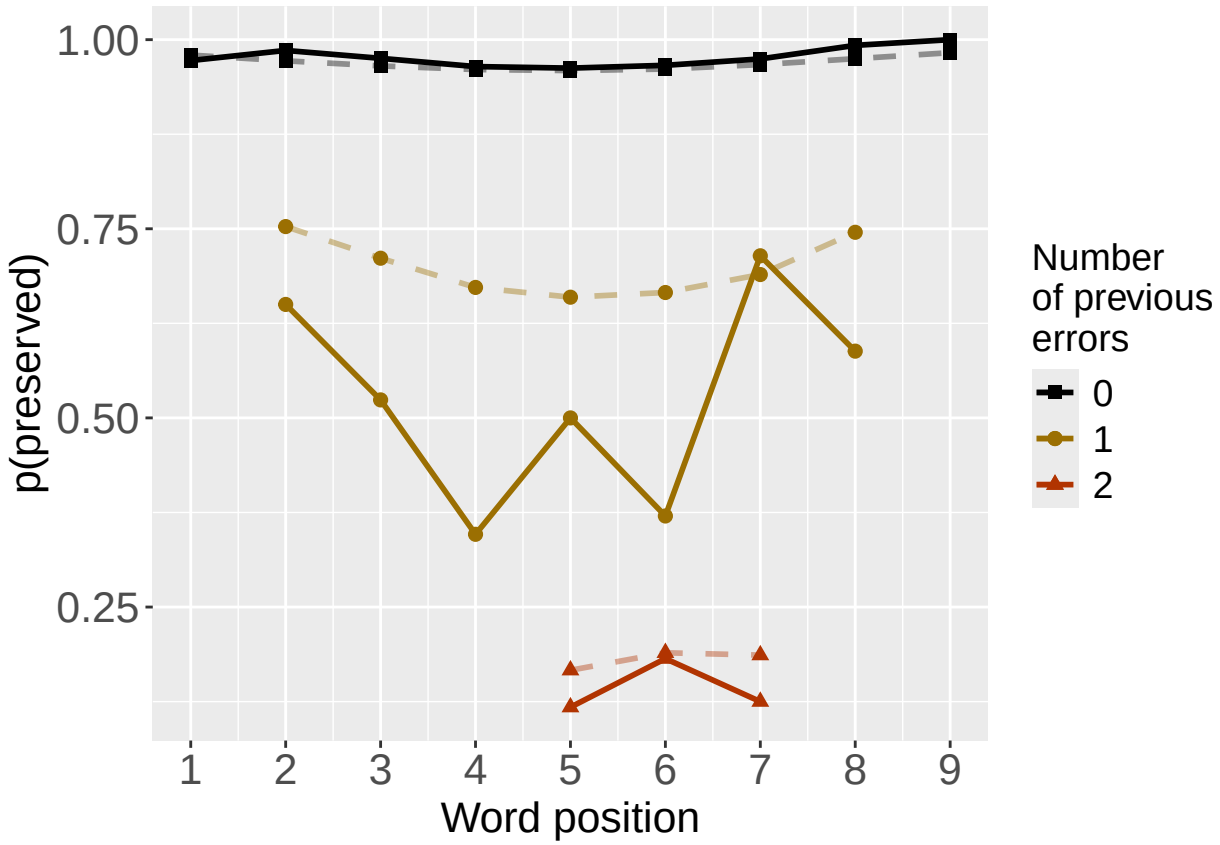
```
print(PrevCorPlot)
```




```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```




```
PlotName<-paste0(FigDir,CurPat,"_",
                 RunLabel,"_",CurTask,
                 "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression =
```

```
## Saving 6.5 x 4.5 in image
```

```
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "
```

```
## Saving 6.5 x 4.5 in image
```

```
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
  # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
                        family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
```

```
  }
  ModelNames<-c(paste0("***",BestModelFormulaL3),
                rep(BestModelFormulaL3Rnd,RandomSamples))
  AICValues <- c(BestModelL3$aic,RndModelAIC)
  BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
  BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
  BestModelRndDF <- rbind(BestModelRndDF,
                          data.frame(Name=c("Random average"),
                                     AIC=c(mean(RndModelAIC))))
  BestModelRndDF <- rbind(BestModelRndDF,
                          data.frame(Name=c("Random SD"),
                                     AIC=c(sd(RndModelAIC))))
  write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                  "_best_model_with_random_cum_term.csv"),
            row.names = FALSE)
}
```

```
# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)
```

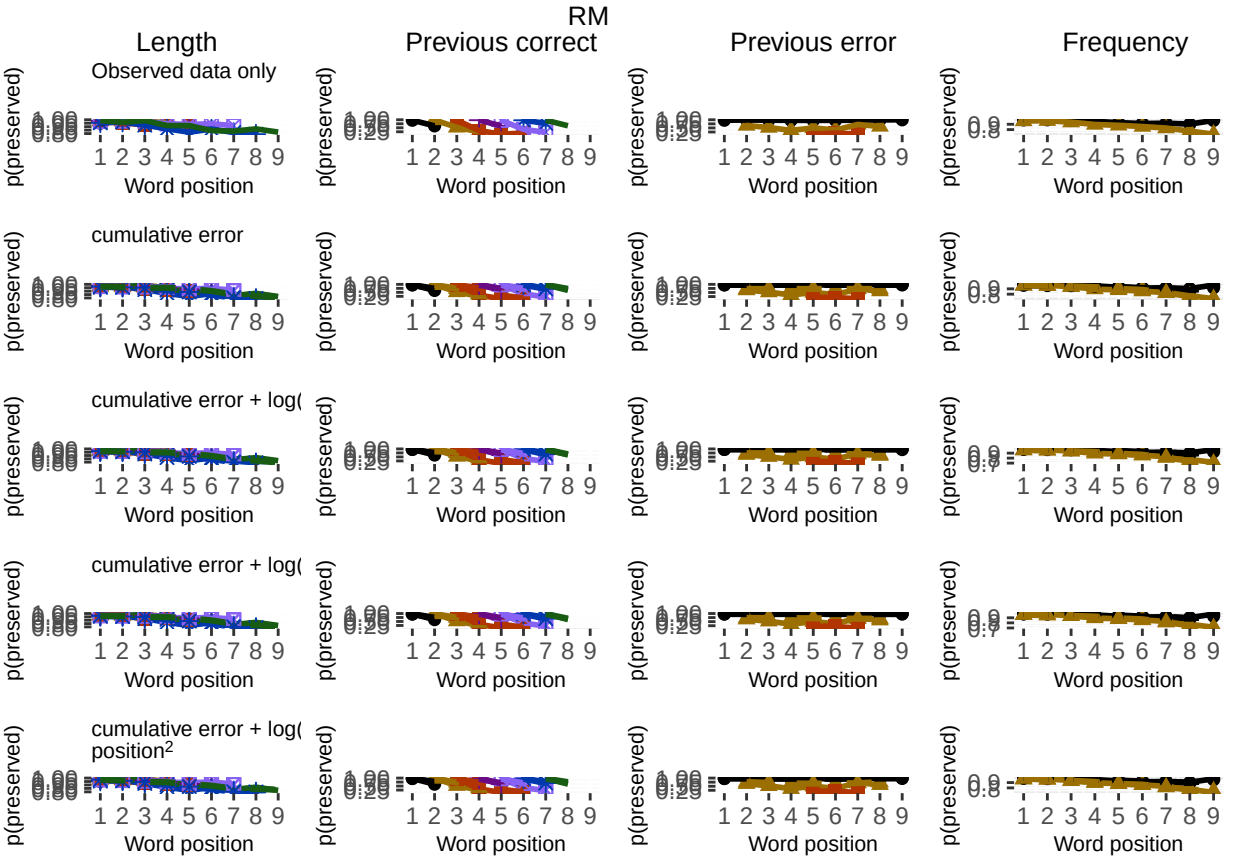
```
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.390       -2.459
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4396 Residual
## Null Deviance:       2237
## Residual Deviance: 1442  AIC: 1446
## log likelihood:  -721.1634
## Nagelkerke R2:  0.4145919
## % pres/err predicted correctly:  -345.2065
## % of predictable range [ (model-null)/(1-null) ]:  0.3974615
```

```
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr     log_freq          pos
##     3.50768     -2.37653      0.15164     -0.02134
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4394 Residual
## Null Deviance:       2237
## Residual Deviance: 1427  AIC: 1435
## log likelihood:  -713.6139
## Nagelkerke R2:  0.4217676
## % pres/err predicted correctly:  -345.3831
## % of predictable range [ (model-null)/(1-null) ]:  0.3971541
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr     log_freq
##      3.4281      -2.4008       0.1543
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4395 Residual
## Null Deviance:       2237
## Residual Deviance: 1428  AIC: 1434
## log likelihood:  -713.7798
## Nagelkerke R2:  0.4216102
## % pres/err predicted correctly:  -345.3721
## % of predictable range [ (model-null)/(1-null) ]:  0.3971732
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr     log_freq          pos     I(pos^2)
##     4.31510     -2.38121      0.15653     -0.47918      0.05087
##
## Degrees of Freedom: 4397 Total (i.e. Null);  4393 Residual
## Null Deviance:       2237
## Residual Deviance: 1418  AIC: 1428
## log likelihood:  -708.7628
## Nagelkerke R2:  0.4263655
## % pres/err predicted correctly:  -343.5115
## % of predictable range [ (model-null)/(1-null) ]:  0.4004114
## ***************************
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
```

```
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
```

```
ggsave(paste0(PlotName,".tif"),plot=FactorPlot,width = 360,height=400,units="mm",device="tiff",compressi
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot
```




```
DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                       CurTask,"_dominance_analysis_table.csv"),
          row.names = TRUE)
kable(DAContributionAverage)
```

	CumErr	I(pos^2)	pos	log_freq
McFadden	0.3151190	0.0172896	0.0207271	0.0131638

	CumErr	I(pos^2)	pos	log_freq
SquaredCorrelation	0.1448506	0.0085535	0.0102437	0.0063331
Nagelkerke	0.1448506	0.0085535	0.0102437	0.0063331
Estrella	0.1798839	0.0090918	0.0109041	0.0071904

	deviance	deviance_explained
CumErr + log_freq + pos + I(pos^2)	1417.526	819.3757
CumErr + log_freq + pos	1427.228	809.6735
CumErr + log_freq	1427.560	809.3417
CumErr	1442.327	794.5746
null	2236.901	0.0000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                                                 model deviance deviance_explained percent_explained
## CumErr + log_freq + pos + I(pos^2) CumErr + log_freq + pos + I(pos^2) 1417.526           819.3757          36.62995
## CumErr + log_freq + pos                       CumErr + log_freq + pos 1427.228           809.6735          36.19621
## CumErr + log_freq                                   CumErr + log_freq 1427.560           809.3417          36.18138
## CumErr                                                         CumErr 1442.327           794.5746          35.52122
## null                                                             null 2236.901             0.0000           0.00000
##                                     percent_of_explained_deviance increment_in_explained
## CumErr + log_freq + pos + I(pos^2)                     100.00000             1.18410553
## CumErr + log_freq + pos                                 98.81589             0.04048829
## CumErr + log_freq                                       98.77541             1.80224192
## CumErr                                                  96.97316            96.97316425
## null                                                          NA             0.00000000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
```

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumErr + log_freq + pos + I(pos^2)	36.62995	100.00000	1.1841055
CumErr + log_freq + pos	36.19621	98.81589	0.0404883
CumErr + log_freq	36.18138	98.77541	1.8022419
CumErr	35.52122	96.97316	96.9731643
null	0.00000	NA	0.0000000

```
                                 "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents
```

```
##            Nagelkerke
## CumErr     0.85215862
## I(pos^2)   0.05032021
## pos        0.06026365
## log_freq   0.03725752
```

```
sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)
```



```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
sse_table<-sse_results_table(sse_results_list)
write.csv(sse_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                           CurTask,"_sse_results_table.csv"),row.names = TRUE)
sse_table
```

```
##                                    model p_accounted_for model_deviance diff_CumErr+log_freq+pos   diff_CumErr diff_CumErr+log_freq
## 1          preserved ~ CumErr+log_freq+pos       0.9324579       1427.228              0.000000000 -0.0020531317        -0.0023732505
## 2                      preserved ~ CumErr       0.9345111       1442.327              0.002053132  0.0000000000        -0.0003201188
## 3              preserved ~ CumErr+log_freq       0.9348312       1427.560              0.002373250  0.0003201188         0.0000000000
```

model	p_accounted_for	model_deviance
preserved ~ CumErr+log_freq+pos	0.9324579	1427.228
preserved ~ CumErr	0.9345111	1442.327
preserved ~ CumErr+log_freq	0.9348312	1427.560
preserved ~ CumErr+log_freq+pos+I(pos^2)	0.9455507	1417.526

model	diff_CumErr+log_freq+pos	diff_CumErr	diff_CumErr+log_freq	diff_CumErr+log_freq+pos+I(pos^2)
preserved ~ CumErr+log_freq+pos	0.0000000	-0.0020531	-0.0023733	-0.0130928
preserved ~ CumErr	0.0020531	0.0000000	-0.0003201	-0.0110397
preserved ~ CumErr+log_freq	0.0023733	0.0003201	0.0000000	-0.0107195
preserved ~ CumErr+log_freq+pos+I(pos^2)	0.0130928	0.0110397	0.0107195	0.0000000

```
## 4 preserved ~ CumErr+log_freq+pos+I(pos^2)       0.9455507       1417.526              0.013092791  0.0110396592          0.0107195404
##   diff_CumErr+log_freq+pos+I(pos^2)
## 1                       -0.01309279
## 2                       -0.01103966
## 3                       -0.01071954
## 4                        0.00000000
```



```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                   CurTask,"_results_report_df.csv"),row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```